

Entrega: A entrega da resolução é realizada na página da disciplina no Moodle juntando, num único ficheiro compactado, o código fonte, o Makefile.

Objetivos: Familiarização com o ambiente UNIX/LINUX; Conceção de programas baseados no paradigma cliente/servidor utilizando *sockets* como mecanismo de comunicação entre processos; Conceção de programas concorrentes com base em múltiplas tarefas; Consolidação da programação ao nível de sistema.

I. Realize os seguintes programas

1. Realize um programa servidor e o respetivo cliente com as seguintes características:

- O servidor disponibiliza um conjunto de serviços. Cada serviço é implementado internamente através da execução de um programa externo. Considere os seguintes serviços:

CPUIINFO Obtenção das características do processador através da execução do comando `lscpu`

MEMINFO - Obtenção da memória do sistema através da execução do comando `free -h`

- Protocolo de comunicações binário a respeitar:

Pedido do cliente: o cliente envia 1 byte (em C tipo `uint8_t`) com o código do serviço pretendido:

Código	Serviço
1	<code>lscpu</code>
2	<code>free -h</code>

Resposta do servidor:

1. O servidor começa por enviar ao cliente 1 byte (em C `uint8_t`) com o estado do pedido(status):

Valor do estado (status)	Significado
0	Ok
1	Serviço inválido (código de serviço pedido inválido)
2	Erro na execução do serviço

2. Caso `status = 0`

O servidor envia o conteúdo do `stdout` numa sequência de blocos com o seguinte formato:

[4 bytes de dimensão dados][dados]

Em que a dimensão é um valor `uint32_t` indicando o número de bytes dos dados e os dados contém exatamente esse número de bytes

O fim da sequência de blocos é indicado por um bloco de dimensão igual a zero: [dimensão dados=0]

Caso `status = 1` O servidor não envia mais nada

Caso `status = 2`

O servidor envia uma mensagem de erro no formato [4 bytes de dimensão dados][dados]

- NOTA:** Não utilizar as funções `system`, `popen`, etc. Utilize apenas as chamadas de sistema estudadas na UC.

a) Realize o servidor e respetivo cliente com base em **sockets streams no domínio UNIX**.

b) Realize um servidor e cliente, idênticos aos da alínea anterior, mas utilizando **socket stream no domínio Internet**.

c) Realize uma nova versão do servidor, **num único processo**, que fique disponível, simultaneamente, num **socket stream no domínio UNIX** e num **socket stream no domínio Internet**.

d) Disponibilize o servidor na máquina Linux remota, de apoio à unidade curricular, utilizada na primeira aula de laboratório e com acesso a partir da rede do ISEL no endereço 10.64.137.7.

2. Na realização do 1º trabalho prático desenvolveu um programa, baseado em múltiplos processos, para determinar o maior, o menor e a soma de todos os elementos num vetor com o objetivo de reduzir o tempo de necessário de processamento. Realize uma nova versão baseada em múltiplas tarefas POSIX seguindo a mesma estratégia de divisão do processamento. Compare esta versão com a baseada em múltiplos processos e a versão sequencial.
3. Desenvolva um programa servidor que presta o serviço de estatística sobre vetores. O serviço consiste num processo servidor, a executar numa máquina UNIX acessível através de uma rede TCP/IP. Desenvolva, também, o respetivo programa cliente que se liga ao servidor, submete o vetor e recebe os resultados. Considere os seguintes pontos:

- O protocolo a utilizar será baseado no definido na 1ª questão. O cliente envia o vetor para servidor como uma sequência de blocos no formato:

[4 bytes dimensão em bytes][dados]

Em que a dimensão é um valor `uint32_t` indicando o número de bytes dos dados e os dados contém uma sequência de valores do vetor representados por valores `uint16_t`

O fim do envio do vetor é assinalado com um bloco de dimensão zero:

[dimensão em bytes = 0]

- O processamento do vetor, no servidor, é realizado com recurso a múltiplas tarefas POSIX como realizado na 2ª questão deste trabalho.
- O servidor começa por enviar ao cliente 1 byte (em C `uint8_t`) com o estado (`status`) do pedido:

Valor do estado (<code>status</code>)	Significado
0	Pedido processado com sucesso
1	Pedido inválido: pedidos que não respeitam o protocolo, i.e., blocos com dimensão inválida, pedido truncado ou vetor vazio
2	Erro de processamento: falha interna do servidor ocorrida após a validação do pedido, nomeadamente falhas de memória ou falhas na criação de tarefas de processamento

Em caso de sucesso (`status = 0`) o servidor responde com:

[2 bytes min][2 bytes max][8 bytes soma]

correspondendo, respetivamente, aos valores mínimo, máximo e soma dos elementos do vetor.

Em caso de erro o servidor envia uma mensagem de erro no formato:

[4 bytes dimensão][dados]

- O servidor deve ser estruturado de forma concorrente usando **uma tarefa POSIX dedicada a cada ligação com um cliente**.
- O cliente deve permitir indicar, pela linha de comandos, o endereço IP do servidor, o porto de ligação e o número de elementos do vetor a enviar.
- A sua implementação deve suportar as comunicações entre os clientes e servidores de forma independente da arquitetura das máquinas.
- Considere na sua implementação a característica dos *sockets streams* não preservarem a fronteira das mensagens.
- Crie um cliente que tenha a opção de realizar várias ligações simultâneas ao servidor. O número de ligações deve ser recebido por argumento da linha de comandos. Esta funcionalidade tem por objetivo testar o servidor aumentando a tensão (*stress*) na utilização do serviço prestado.
- Disponibilize o servidor na máquina Linux remota, de apoio à unidade curricular.
- Esta proposta de trabalho deixa em aberto algumas questões que constituem opções a serem tomadas pelos alunos. Os testes realizados para verificar a correção do trabalho constituem, também, um ponto de avaliação.

II. Indique se as seguintes afirmações são verdadeiras (V) ou falsas (F)

4. Na avaliação a resposta certa será cotada com 100%, resposta errada -25%

a)

A utilização da chamada de sistema <code>fork()</code> permite a execução de dois troços de código concorrentes com espaços de endereçamento separados.	
A utilização da chamada de sistema <code>pthread_create()</code> permite a execução de dois troços de código concorrentes com espaços de endereçamento separados num mesmo processo.	
A chamada de sistema <code>exec1p()</code> cria um processo.	
A chamada de sistema <code>pthread_join()</code> permite esperar pela terminação da tarefa identificada no primeiro argumento da função.	

b)

O mecanismo de comunicação <i>fifo</i> (<i>named pipe</i>) apenas funciona entre processos com grau de parentesco.	
Os <i>sockets</i> do domínio UNIX e os <i>fifos</i> são identificados através de um ficheiro especial no sistema de ficheiros.	
Nos <i>sockets stream</i> a função <code>bind</code> serve para associar o <i>socket</i> a uma tarefa	
Os <i>sockets</i> são representados ao nível do núcleo do sistema operativo como um tipo de ficheiro e podem ser usados para o redireccionamento de I/O e a receção e envio de dados realizados através das funções de <code>read()</code> e <code>write()</code> .	

c)

A ordem de execução entre as tarefas criadas através de <code>pthread_create()</code> num processo é garantida pelo sistema operativo.	
Uma tarefa criada no estado <code>detach</code> , quando terminar os recursos associados a essa tarefa serão, automaticamente, libertados.	
Uma tarefa criada no estado <code>joinable</code> , quando terminar os recursos associados a essa tarefa serão, automaticamente, libertados.	
Cada tarefa criada com <code>pthread_create()</code> corre obrigatoriamente num processador diferente.	

- d) Indique, para cada uma das afirmações, se a utilização de múltiplas tarefas, num programa a executar-se num sistema operativo UNIX, é uma razão válida:

Para dividir o processamento por ações concorrentes de forma a maximizar a utilização de toda a capacidade de processamento do <i>hardware</i> .	
Para poder executar dois programas (ficheiros executáveis) diferentes em concorrência e de uma forma mais rápida.	
Para poder realizar operações I/O em simultâneo com outras operações num mesmo processo.	
Para ter a execução de dois troços de código concorrentes com espaços de endereçamento separados num mesmo processo.	

Bom trabalho,
Nuno Oliveira